

📄 粘贴的文本 (1).txt

文档

根据来源中的outmap.txt；逐行解释附件log中的内容与outmap.txt代码的对应

下面按**“日志 → 对应代码变量/逻辑 → 含义”的方式，一步一步对齐说明。先只做核心主链路（停车位→边界→终点）**，确保你理解正确，再可以继续细化其它模块。

一、核心代码（outmap.txt）在做什么

先抽取关键变量（这是后面所有 log 的“源头”）：

⌕ C++

Obj2Pt = APAMap_GInfo.SlotPar.SlotBordPt[0];
Obj1Pt = APAMap_GInfo.SlotPar.SlotBordPt[1];
SlotLen = APAMap_GInfo.SlotPar.SlotLen;
EndPos = APAMap_GInfo.SlotPar.EndPos;

OrgAng = APAMap_GInfo.NewCordSysAng;
OrgPt = APAMap_GInfo.NewCordSysOPt;

bSlotDataAtRigthSide = APAMap_GInfo.SlotPar.bSlotDataAtRigthSide;

CurCarCoordinateX = APAMap_GInputData.CarLocInfo.CarPos.Coordinate.x * 0.001;
BloundaryOffsetY = 0;
bSeizeEndCarPosFlag = FALSE;
SafeDis = 250;

📄

👉 这段的本质是：

变量	含义
Obj1Pt / Obj2Pt	车位两侧边界点
SlotLen	车位长度
EndPos	规划出的停车终点
OrgPt / OrgAng	新坐标系
bSlotDataAtRigthSide	车位在车左还是右
CurCarCoordinateX	当前车位置
SafeDis	安全距离

二、日志逐行对齐（第一阶段：输入与车位）

📄 输入数据

==INPUTDATA==TOTALMAP==...carpose: 345.757355, -7923.585851, 0.153938

📄

📄 粘贴的文本 (1)

👉 对应代码：

⌕ C++

CurCarCoordinateX = CarPos.x * 0.001;

📄

👉 含义：

- 车辆当前位姿 (x,y,θ)
- 后续所有计算的基准

📄 车位点（非常关键）

==ParkOutSlotInfoFromTotalMap==SlotPt:
0(3254, -3629), 1(971, -4045), 2(1975, -9466), 3(4258, -9050)

📄

📄 粘贴的文本 (1)

👉 对应代码（间接来源）：

⌕ C++

APAMap_GInfo.SlotPar.SlotBordPt[]

📄

👉 含义：

这是一个矩形车位：

0 ----- 1
| |
| |
3 ----- 2

📄

后面会选：

</> C++

Obj2Pt = SlotBordPt[0]
Obj1Pt = SlotBordPt[1]

👉 log 中后续验证：

ObjPt2(-1947.49, -4560.28)=ObjPt1(1091.15, -3894.80)

📄 粘贴的文本 (1)

✔ 已映射成功

三、第二阶段：选取当前车位（核心逻辑）

==APAMap_ParkingOutBuildCurCarPosSlotByTwoNearbySlot==
Valid==bResult1(1)

📄 粘贴的文本 (1)

👉 对应代码函数（你文件中）：

</> C++

APAMap_ParkingOutGetSlotInfoFromVPLSlotPts(...)

👉 含义：

- 从多个候选车位中
- 选出“当前车辆所属车位”

四、第三阶段：几何计算（最重要）

代码关键：

</> C++

AlgCom_LineParABCbyTwoPoints(...) // 建直线
AlgCom_LineParABCByPerPendLineAndPointOnLine(...) // 垂线
AlgCom_CrossPointOfTwoLines(...) // 求交点

📄 outmap

对应日志

==ObjbyFSDAndOD==
ObjPt2(-1947.49, -4560.28)
ObjPt1(1091.15, -3894.80)
EndPos(0.22, -2.11)
SlotLen(3109.67)

📄 粘贴的文本 (1)

👉 对应关系：

log字段	代码变量
ObjPt2	Obj2Pt
ObjPt1	Obj1Pt
SlotLen	SlotLen
EndPos	EndPos

👉 这里做的事：

用车位边界 → 计算停车终点

五、第四阶段：偏移与安全处理

代码：

</> C++

SafeDis = 250;
BloundaryOffsetY = 0;
bSeizeEndCarPosFlag = FALSE;

对应日志

FSDOffset(...)
ODOffset(...)
OffsetX(0,0)
OffsetY(0,1.39)

📄 粘贴的文本 (1)

👉 含义：

项	含义
FSDOffset	视觉/融合感知偏移
ODOffset	障碍物检测偏移
OffsetX/Y	最终修正
1.39	Y方向修正（贴边/安全距离）

六、第五阶段：终点是否被“侵占”

代码：

</> C++

bSeizeEndCarPosFlag // FSD侵占终点

outmap

对应日志

bSeizeEndCarPosFlag(0)

粘贴的文本 (1)

👉 含义：

- 0 → 终点安全
- 1 → 被占用，需要重新规划

七、第六阶段：边界生成（路径约束）

=BSegData==
=NSegData==

粘贴的文本 (1)

👉 对应代码（隐式）：

- 车位边界
- 可行驶区域

👉 含义：

数据	含义
BSegData	边界线
NSegData	可行驶路径线

八、第七阶段：最终输出

==APAMap=Output==
CarPos(...)
Obj2_x(...)
Obj2_y(...)
endpos(-7.208, -1.580)

粘贴的文本 (1)

👉 对应代码变量：

</> C++

EndPos
Obj2Pt
Obj1Pt

九、总结（主链路）

整个 log 实际就是执行了这条链：



十、关键结论（非常重要）

你现在可以用一个“调试思维”去看 log：

只看这5个字段就够定位问题：

SlotPt

ObjPt1/ObjPt2

EndPos

OffsetX/Y

bSeizeEndCarPosFlag

← 车位是否正确

← 边界是否正确

← 终点是否异常

← 偏移是否异常

← 是否被占用

如果下一步需要

可以继续做更深入的三件事（建议按顺序）：

1. 逐函数级别（精确到每一行代码）对齐 log

2. 分析为什么出现 -1e18 / nan（你日志里很多）

3. 结合路径规划算法（A/RRT）解释边界如何影响轨迹*

只要说“继续第几步”。

来源

第1步

好的，进入**第1步：函数级逐行对齐（严格对应 log）**。
本步只聚焦一个函数，确保完全理解，再进入下一个。

一、目标函数

选择最关键函数：

</> C++

APAMap_ParkingOutGetSlotInfoFromVPLSlotPts(...)

outmap

👉 因为它直接对应日志：

==ParkOutGetSlotInfoFromLocVPLSlotByTotalMap==

粘贴的文本 (1)

二、函数执行链（先建立骨架）

这个函数内部逻辑可以拆成 **5步**：

① 获取输入（车位点、车位置）

② 判断车位左右

③ 选取Obj1/Obj2点

④ 构建三条线（Close / Far / 垂线）

⑤ 求交点 → 修正车位几何

下面逐行对齐。

三、逐行对齐（严格对应 log）

Step 1：输入数据

</> C++

CurCarPos = APAMap_GInputData.CarLocInfo.CarPos;
ParkOutMode = APAMap_GInputData.ParkReqPar.parkoutmode;
bSlotDataAtRigthSide = APAMap_GInfo.SlotPar.bSlotDataAtRigthSide;

👉 对应 log：

CarPos(345.757721, -7923.586914, 0.153938)
parkoutmode(0)
side(0)

粘贴的文本 (1)

👉 含义：

变量	值	说明
CurCarPos	(345,-7923)	当前车
ParkOutMode	0	平行/垂直出库模式
bSlotDataAtRigthSide	0	车位在左

Step 2: 构造车辆矩形

</> C++

APAMap_GetCarRectArea(..., CurCarPos, &pRectPt[0], &pRectLine[0]);

📄

👉 log没有直接打印，但影响后续：

CurCarCoordinateX(0.35)

📄

📄 粘贴的文本 (1)

👉 含义：

- 得到车身四个角点
- 用于判断与车位关系

Step 3: 选择车位角点索引（关键）

</> C++

if (bSlotDataAtRigthSide == TRUE) { Obj2PtIndex = 0; Obj1PtIndex = 1; } else { Obj2PtIndex = 3; Obj1PtIndex = 2; } }

📄

👉 对应 log：

SlotPt:0(971, -4045),1(1975, -9466),2(4258, -9050),3(3254, -3629)

📄

📄 粘贴的文本 (1)

👉 实际选点：

Obj2Pt = SlotPt[3]
Obj1Pt = SlotPt[2]

📄

👉 对应后续 log：

ObjPt2(-1947.49, -4560.28)
ObjPt1(1091.15, -3894.80)

📄

📄 粘贴的文本 (1)

✓ 完全对应

Step 4: 构建“近边线”(CloseLine)

</> C++

AlgCom_LineParABCbyTwoPoints(pVPLSlotPts[Obj1PtIndex], pVPLSlotPts[Obj2PtIndex], &CloseLine);

📄

👉 对应 log（隐式）：

SlotBordPt(...)

📄

📄 粘贴的文本 (1)

👉 含义：

- 用 Obj1 → Obj2 构建车位入口边

Step 5: 构建“远边线”(FarLine)

</> C++

AlgCom_LineParABCbyTwoPoints(pVPLSlotPts[Obj1InnerIndex], pVPLSlotPts[Obj2InnerIndex], &FarLine);

📄

👉 对应 log：

SlotLen(3109.67)

📄

📄 粘贴的文本 (1)

👉 含义：

- 车位另一侧边界
- 用于计算深度

Step 6: 构建垂线（核心几何）

</> C++

Templine = AlgCom_LineParABCBYPendLineAndPointOnLine(
 pVPLSlotPts[Obj2PtIndex],
 &CloseLine);

📄

- 👉 含义：
- 从 Obj2 点做一条 垂直于入口边的线

👉 对应 log（间接）：

ObjAng(13.79)
OrgAng(102.35)

📄

📄 粘贴的文本 (1)

👉 本质：

👉 在算车位方向角（法向）

Step 7：求交点（最关键）

</> C++

AlgCom_CrossPointOfTwoLines(
 &Templine,
 &FarLine,
 &pVPLSlotPts[Obj2InnerIndex]);

📄

👉 对应 log：

NearByCarPosSlot1(...)
NearByCarPosSlot2(...)

📄

📄 粘贴的文本 (1)

- 👉 含义：
- 修正车位矩形（防止畸形）
 - 得到标准矩形

四、函数输出（核心结果）

这个函数最终输出：

</> C++

*pObj2Pt
*pObj1Pt
*pObj2Ang
*pObj1Ang
*pObj2Dis
*pObj1Dis
*pCarOffsetX
*pMinSlotDpth
*pVPLSlotDpth

📄

对应 log

Obj2Pt(-1517, -4466)
Obj1Pt(798, -3959)
ObjAng(13.79)
Dis(-41.94, 124.09)
CarOffsetX(-56.45)
MinSlotDpth(4919.66)
VPLSlotDpth(5557)

📄

📄 粘贴的文本 (1)

五、这一函数本质（总结一句话）

把原始车位点 → 转换成“标准矩形 + 几何参数”

📄

输出的是：

- 边界点（Obj1/Obj2）
- 朝向（角度）
- 深度（slot depth）
- 偏移（CarOffset）

六、你现在应该能做到

看到任何一行 log：

ObjPt2(...)

📄

能立即知道：

来源 = SlotPt[i]
经过 = 索引选择 + 坐标变换 + 直线求交

📄

七、下一步建议（按优先级）

如果继续深入，建议：

👉 第2步（强烈推荐）

分析异常值来源：

-1e18 / nan / 32767

📄

这些在你 log 里大量出现，是关键问题。

如果继续，请直接说：

👉 「第2步」

来源

📄 👍 🔖 🔄 ↺ ...